BIASHARA MALL

The Biashara Mall Codebase — a guided tour

Who this is for: anyone who needs to understand how the code fits together before changing it — including us, in six weeks. `BUILD_LOG.md` says *what happened and why*, chronologically. This says *what exists now and how it works*. Current as of 2026-08-25 · **605 tests** · 28 test files · 17 migrations **This document was rewritten after the 2026-08-22 pivot and again after the 2026-08-25 buyer-surface change.** If something here contradicts an older PDF in `docs/`, this is right and the PDF is a snapshot of a direction we left.

---

1. The thirty-second version

A Kenyan seller already runs a catalogue inside WhatsApp groups. They forward a post to our bot; an AI reads the photo and drafts a product; the seller confirms a price and opens their shop. Buyers reach that shop **inside WhatsApp** and pay the seller directly on M-Pesa.

Two hard problems shape everything else.

The price is not in the caption. Measured against 24 real captions: zero mention KSh, three contain any price-like number. Sellers withhold it deliberately — that withholding is the buyer bottleneck this product removes. So the price is read from the image, and when it is not there, the seller supplies one number by hand.

We cannot control which browser a link opens. Tested on a real handset: a plain URL and a native CTA button **both** left WhatsApp for Chrome. So the buyer surface is not a page we link to — it is the conversation itself.

```
 13,594 lines   application code   (70 Python files)
  3,695 lines   templates          (28 Jinja2 files)
  8,806 lines   tests              (605 tests, 28 files)
     17         migrations         (each reviewed line by line before applying)
     52         HTTP routes
```

---

2. Running it

```
cd backend
.venv\Scripts\activate                # Windows

uvicorn app.main:app --reload         # http://localhost:8000
pytest                                # 605 tests
ruff check . && mypy app tests        # the quality gate
alembic upgrade head                  # apply migrations

python scripts/seed_test_shop.py      # stock to look at, no Apify needed
```

Two databases are required and they must differ — see §8.

> **A warning worth its own line.** `--reload` was observed *silently not reloading* on this machine, and a killed uvicorn parent can leave a live worker holding the port. A server that lies about which code it runs costs hours. If behaviour disagrees with the source, kill every `uvicorn` process — **children first** — confirm the port is free, and start one fresh.

3. The layering rule

```
HTTP request  /  inbound WhatsApp message
     │
     ▼
  api/          parse, validate, delegate, render.  NO business logic.
     │
     ▼
  services/     the rules. Providers hidden behind our own adapters.
     │
     ▼
  models/       persistence, and the constraints that make bad states impossible.
```

`agent/` sits beside `services/` and is the **only** place a model provider SDK is imported. A route containing business logic is in the wrong layer; so is a service that knows it is talking to Gemini.

Routes commit; `get_db` does not. This bit us once, badly — sixteen mutating routes had no `db.commit()` and nothing persisted, while the whole test suite stayed green because fixtures roll back inside a transaction. See §7.

---

4. File by file

The spine

FILE	WHAT IT OWNS
`main.py`	Wires routers. Owns nothing else. Registers the `LoginRequired` redirect and the HTML-404 handler.
`config.py`	Typed settings. **Nothing else reads `os.environ`.** Trims whitespace off pasted secrets — see §7.
`db.py`	Engine and session. Import `get_db`; never build an engine. `autoflush=False`, which matters more than it sounds — see §7.
`dependencies.py`	`current_account`, `current_seller`, `optional_account`.
`security.py`	Argon2 hashing, session tokens, phone-proof tokens.
`secrets_vault.py`	Fernet encryption for sellers' Daraja credentials. Deliberately *not* in `security.py`: hashing is one-way, this is the opposite.
`templating.py`	The one Jinja environment, and its filters (`media`, `media_abs`, `initials`).

The AI

FILE	WHAT IT OWNS
`agent/draft.py`	The vision agent. `draft_from_forwarded(caption, image)` is the live path; `draft_from_cover` / `draft_from_video` belong to the parked scraper. **The only file that knows which vision provider we use.**
`schemas/draft.py`	`ProductDraft` — the Pydantic guardrail on model output.
`services/intake.py`	Forwarded post → download → parse-once → DRAFT product.
`services/drafting.py`	The cost-ordered cascade. Parked with the scraper.
`models/parsed_media.py`	One row per image we have paid to read. The cache that makes "once ever" a database fact.

WhatsApp

FILE	WHAT IT OWNS
`api/webhooks.py`	`POST /webhooks/whatsapp`. Verifies Twilio's signature, dedupes by `MessageSid`, replies in TwiML.
`services/bot.py`	The conversation. Buyer browsing and checkout; seller forwarding.
`services/messaging.py`	Outbound sending, behind a `Messenger` Protocol. Plain httpx, no SDK.
`models/wa_message.py`	Every inbound message, raw. Unique on `provider_message_id`.
`models/wa_conversation.py`	Where a buyer has got to, and which basket is theirs.

Commerce

FILE	WHAT IT OWNS
`services/catalogue.py`	The publish gate. Refuses without a price, and names the item when it does.
`services/cart.py`	Server-side baskets, scoped to exactly one seller.
`services/orders.py`	`place_order`, `claim_payment`, `confirm_payment`, `cancel_order`.
`services/payments.py`	Idempotent Daraja callback handling.
`services/daraja.py`	STK push, behind an `StkEngine` adapter.
`services/storefront.py`	The public catalogue, preview rules, link-preview cover.
`services/customers.py`	Buyers derived from orders. **There is no customer table** — see §5.

The workspace

Server-rendered Jinja behind a login wall. `app_base.html` is the shell; `partials/icons.html` is the **one** icon set; `static/css/workspace.css` holds everything used by more than one page. A rule earns its place there once a second page needs it.

Pages: dashboard, products, product_new, orders, order_detail, money, payment_settings, customers. Plus `accounts` / `connect` / `claim`, which belong to the parked TikTok flow and are off the main nav.

5. The data model

The two identities

An **Account** signs in. A **Seller** is the shop. `Account.email` is nullable — phone-and-OTP is the real login, because the WhatsApp number is the identity a Kenyan micro-seller actually has. A CHECK constraint requires one or the other.

Provenance is two dimensions

`platform` (where it came from) and `ingest_method` (how it arrived) are separate columns with a constraint requiring them to agree. A forwarded post is `manual` + `upload` and carries no `platform_post_id`, so a future feed sync can never overwrite something a seller sent by hand.

Customers are derived, not stored

A buyer never signs up — that friction is what the product removes — so there is no moment at which a customer row could be created and nothing on one to edit. `services/customers.py` groups orders **by phone**, because the same person is "Akinyi", "akinyi o" and "Akinyi Otieno" across three orders and grouping by name would show a seller three customers where they have one.

The rails — constraints in Postgres, not in service code

These are the ones that have actually caught bugs:

CONSTRAINT	WHAT IT PREVENTS
`ck_sellers_published_needs_whatsapp`	A live shop nobody can contact. Caught a test that tried to publish without a number.
`ck_payment_methods_pochi_has_no_stk`	Pretending Pochi can take an STK push. It cannot — Daraja does Paybill and Buy Goods only.
`ck_orders_subtotal_positive`	An order with zero totals. Caught 20 tests when totals were computed after the row was created.
`ck_orders_paid_has_timestamp`	A paid order nobody can audit.
`ck_wa_conversations_paying_needs_order`	An M-Pesa code arriving with nowhere to land.
`ck_parsed_media_draft_xor_error`	A cache row recording nothing, re-parsed forever.
unique `provider_message_id`	One forwarded post becoming two products when Meta redelivers.
unique `provider_media_id`	Paying twice to read the same image.

Money

Integer KES throughout. **Order lines copy the price**; they never join to a live product, so a seller editing a price cannot rewrite what a buyer already paid. Carts do the opposite — they read through to the live product, because a rename mid-basket should be visible.

`pending → awaiting_confirmation → paid`. Only the seller makes the last move on the manual path. **A buyer-entered M-Pesa code is a claim, not a payment** — `claimed_code` and `mpesa_receipt` are deliberately separate columns.

6. Three flows, traced

A. A seller forwards a catalogue post

```
photo + caption ──▶ POST /webhooks/whatsapp
                          │  signature verified against APP_BASE_URL
                          │  MessageSid deduped
                          ▼
                    services/bot.handle(media=[...])
                          │  find_seller_by_phone → this is a SELLER
                          ▼
                    services/intake.ingest_forwarded_post
                          │  download (authenticated)
                          │  parse_once  ── cached by media id ──▶ ParsedMedia
                          │  agent.draft_from_forwarded
                          ▼
                    DRAFT product, never published
                          │
                          ▼
        "Added *Ankara Shirt* to your drafts. I couldn't see a price…"
```

The agent proposes a name, description and — only if it can literally see one — a price. There is no estimate and no fallback: a wrong price reaches a buyer, a missing one costs the seller five seconds.

B. A buyer shops, inside WhatsApp

```
wa.me/<bot>?text=shop <slug>     ← a DEEP LINK, not a URL. Opens WhatsApp itself.
        │
        ▼
  categories ▸ products ▸ product+photo ▸ add ▸ cart ▸ checkout
        │
        ▼
  place_order  ── same service the web checkout calls ──▶ Order
        │
        ▼
  "Send KSh 950 to 07XX. Reply with the M-Pesa code."
        │
        ▼
  claim_payment  → awaiting_confirmation → the seller confirms in the workspace
```

State lives in `WaConversation`, because a bare "2" means nothing without what we last asked. The basket token lives there too — see §7.

C. A seller opens their shop

`/products` shows the review queue, **lowest parse confidence first**: a wrong price hides in the drafts the model was least sure about, and a seller working a date-ordered list meets those last, after the habit of clicking Publish has formed.

`publish_product` refuses without a price. Opening the shop is a separate, deliberate act — and until it happens, `/shop/<slug>` 404s for everyone except the owner, who sees a preview bar over the real page.

7. Five lessons the code carries

1. Nothing persisted, and 465 tests said it was fine. Sixteen mutating routes never called `db.commit()`. Test fixtures run inside a transaction that is rolled back, so they *structurally cannot* catch it. A test that commits and re-reads from a separate session is the only kind that would.

2. A field nothing writes. `whatsapp_number` was read everywhere and written nowhere, so no seller could ever open a shop. The test factory set it, which hid the hole completely.

3. `get_or_create_cart` mints a new token. The web flow relies on that and writes it back to a cookie. A chat has no cookie, so the first bot version passed a made-up token that was never found — every message got a fresh empty basket. It said "Added ✅" and then "your basket is empty".

4. `autoflush=False`. The parse cache added its row but never flushed it, so the lookup at the top of the same function never saw it and every redelivery paid Gemini again. Exactly the bug the cache exists to prevent, shipped inside the cache.

5. Whitespace on a pasted secret is invisible and fatal. A `DATABASE_URL` of `' \n'` cost a deploy. The Twilio auth token is worse — it is a shared HMAC secret, so one stray byte makes every genuine message look forged. `config.py` now trims all of them.

The through-line: **four of these five were invisible to a green test suite.** Walk the real path against a real database before believing it works.

---

8. Tests

605 tests, 28 files, mirroring `app/`. Transactional fixtures against real Postgres — no SQLite, because the rails in §5 are Postgres CHECK constraints and a database that ignores them tests nothing.

External services are always mocked. Gemini, Twilio, Daraja and Apify never receive a call from the suite. That is not tidiness: a suite that costs money or needs quota is a suite nobody runs.

Two files earn a mention:

- `test_web_workspace.py` renders **every** workspace page in **both** states. Jinja has no compile step, so a renamed context key is a 500 on a page a seller opened — caught by nothing else.
- `test_bot.py::TestTheBasketSurvives` exists because lesson 3 above shipped.

Two databases, on purpose

`TEST_DATABASE_URL` must never equal `DATABASE_URL`; the suite refuses to run if it does. The guard is not paranoia — the suite calls `drop_all`, and it has already prevented one production wipe.

---

9. What exists, and what doesn't

Works today

- Seller auth by phone + OTP; the whole workspace; the publish gate
- The storefront, cart and checkout on the web, with owner preview
- Orders, manual M-Pesa confirmation, idempotent Daraja callback
- Inbound WhatsApp: signature verification, dedupe, conversation

- Buyer shopping in chat; seller forwarding a post into a draft product

Exists but parked

- TikTok scraping (Apify), account verification by bio code, the video tier of the cascade. The adapter seams remain, so it can return without a rewrite.

Does not exist yet

- **The reply loop.** The bot asks for a price; nothing handles the answer.
- **Anthropic.** There is no `anthropic_api_key` setting. The Claude-reasons / Gemini-localises split is a plan, not code.
- **Natural onboarding.** A new seller still gets "open a shop's link".
- **A production WhatsApp sender.** Everything runs on the Twilio sandbox, which requires every recipient to send `join <code>` first. **No real buyer will ever do that** — Meta Business verification is the gate on launching at all.
- **WhatsApp Flows.** The preferred buyer surface, blocked on verification. The chat flow is what exists in the meantime; the storefront is the seller's preview and the desktop fallback.

---

10. Where to start reading

1. `CLAUDE.md` — the direction and the rules that must not be silently changed
2. `services/intake.py` — the product's central promise, in one file
3. `services/bot.py` — the buyer and seller conversations
4. `models/order.py` — the money rails, and the comments explaining each
5. `services/catalogue.py` — the publish gate
6. `docs/BUILD_LOG.md` — why any of the above is shaped the way it is
